

Project Plan and Methodology

OptiEDT

AI-assisted generation, ranking and explanation of university timetables

Author	Mohamed Jawad Tourir		
Academic year	2025 – 2026	Duration	4 weeks
Project	Company internship project	Date	23 July 2026
Field of application	Tunisian public universities (LMD system)		

Contents

1. Introduction.....	4
1.1 Purpose of the document.....	4
1.2 Position in the documentation.....	4
2. Study of the problem	4
2.1 Understanding of the timetabling problem.....	4
2.2 The particular case of the LMD system	4
2.3 Rules specific to the institution	5
2.4 Existing work	5
2.5 The choice which follows the generation.....	6
3. Study of the datasets.....	6
3.1 Approach followed.....	6
3.2 Datasets identified	7
3.3 Problems found during the verification.....	7
3.4 Why the datasets cannot be merged	7
3.5 Why a generated dataset is necessary.....	8
3.6 Method of generation and content obtained.....	8
3.7 Role finally given to each source.....	9
3.8 Data required by the adjustment of the weights.....	9
4. Choice of the resolution method	10
4.1 Nature of the problem	10
4.2 Criteria of evaluation	10
4.3 Approaches studied.....	10
4.4 Comparison and decision.....	11
4.5 Choice of the library and consequences on the solving.....	11
4.6 Method for the ranking of the valid timetables	13
4.7 Place of the learning.....	14
4.8 Place of the language model.....	15
4.8.1 Why scoring and ranking stay outside the model.....	15

4.8.2 Why natural-language constraint entry stays outside	15
4.8.3 Grounding	16
4.9 Recommendation-driven regeneration	16
5. Choice of the technical architecture	17
5.1 Server	18
5.2 Database	18
5.3 Interface	18
5.4 Organisation of the application	18
5.5 Institutional rules held as configuration	18
5.6 Execution of the generation	19
5.7 Isolation of the analysis components	19
6. Development methodology	19
6.1 General approach	19
6.2 Phases of the work	19
6.3 Delivery in two increments	20
7. Validation strategy	21
7.1 Validation on the published instances	21
7.2 Verification of the generated instance	22
7.3 Functional tests	22
7.4 Validation of the analysis layer	22
7.5 Criteria of acceptance	23
8. Planning	23
8.1 Phases and duration	23
8.2 Milestones	24
8.3 Order of reduction in case of delay	24
9. Risks and mitigation	25
10. Deliverables	25

List of figures

Figure 1. Role given to each source and origin of the data used by the application.....	9
Figure 2. Division of the responsibilities between the decision layer and the analysis layer.	12
Figure 3. Sequence from the data to the published timetable, with the three stages of the generation.....	13
Figure 4. AI assistant service, grounded on computed facts	16
Figure 5. AI recommendation and regeneration loop.	17
Figure 6. System architecture and deployment.	17
Figure 7. Sequence of one generation run, from launch to publication.	21
Figure 8. Project workflow, phases of the first increment.	24

List of tables

Table 1. Datasets identified and their content	7
Table 2. Role finally given to each source.....	9
Table 3. Comparison of the resolution approaches.....	11
Table 4. Comparison of the methods for ranking the candidates.....	13
Table 5. Uses of a language model examined.....	15
Table 6. Delivery in two increments.....	20
Table 7. Criteria of acceptance of the validation	23
Table 8. Phases and duration.....	23
Table 9. Milestones of the project.....	24
Table 10. Risks and mitigation.....	25

1. Introduction

1.1 Purpose of the document

This document describes the way the project was studied and the way it will be carried out. It explains how the problem was analysed, how the available data was examined and verified, on what grounds the resolution method was chosen, where artificial intelligence is used and where it is deliberately not used, and how the work is organised over the four weeks of the project. It was written before the beginning of the implementation.

1.2 Position in the documentation

The documentation of the project is made of three documents which follow one another. The present document explains the reasoning and the organisation of the work. The Cahier des Charges states what the application must do. The Software Requirements Specification states how each requirement is expressed in terms of data, interfaces and constraints. Reading them in this order gives the whole progression, from the study of the problem to the technical specification.

2. Study of the problem

2.1 Understanding of the timetabling problem

Building a timetable means giving to each teaching session a moment in the week and a room. The difficulty does not come from the number of sessions but from the links between them. A teacher cannot give two sessions at the same hour. A group of students cannot follow two courses at once. A room cannot receive two sessions simultaneously. Each placement therefore restricts the placements which remain possible for all the others, and a choice which seems correct at the beginning can block the end of the work.

This is what makes manual construction long. The person in charge does not only place sessions, that person verifies after each placement that nothing has been broken elsewhere. When a conflict appears late, part of the work already done has to be undone.

The problem is not merely tedious, it is difficult in a precise sense. University course timetabling contains a graph colouring problem as a particular case, which places it among the problems for which no algorithm is known that solves every instance quickly. This has a practical consequence which the whole project is built around : no honest bound on the solving time can be announced in advance, so the engine must work under a time limit and return the best result obtained when the limit is reached.

2.2 The particular case of the LMD system

In a Tunisian faculty the sessions are not all of the same nature, and this is what makes the problem more difficult than a simple assignment of courses to rooms.

- **CM** : the lecture gathers the whole promotion in a lecture theatre.
- **TD** : the tutorial concerns one group of the promotion, in an ordinary classroom.
- **TP** : the laboratory session concerns a subgroup, in a room equipped for that purpose.

These three levels are linked. The students of a subgroup belong to a group, and the students of a group belong to the promotion. When a lecture is placed, every student of the promotion is

occupied, so no session of any group or subgroup of that promotion can be placed at the same hour. A model which treats the sessions as independent events misses this relation and produces timetables which are impossible in practice.

The instance of this project reflects that structure precisely : 6 promotions, 15 tutorial groups and 30 laboratory subgroups, giving 51 groups in total, and 218 sessions distributed as 32 lectures, 82 tutorials and 104 laboratory sessions. The proportion is characteristic of the LMD organisation and explains why the laboratories are the scarce resource.

2.3 Rules specific to the institution

Beyond the structure of the teaching, an institution applies rules of its own which change the grid of available moments.

- Certain half-days are not used for teaching, and which ones depends on the institution.
- The working day is shortened during a declared period of the year, in particular during Ramadan, whose dates move by about eleven days each year.
- The calendar contains fixed holidays and lunar holidays whose date is known only approximately in advance.
- The promotions of a public faculty are large, which limits the number of lecture theatres able to receive a lecture.

An important decision of design follows from this list, and it is discussed in section 5.5 : these rules are treated as configuration data and not as constraints written into the model.

2.4 Existing work

University timetabling has been studied for a long time and has given rise to international competitions, in particular the competition of 2007, which comprised three tracks : examination timetabling, post-enrolment course timetabling, and curriculum-based course timetabling. The third track defines the problem on which the weekly part of this project is modelled, and its instances are real data coming from a European faculty of engineering. This work was examined for two reasons : it gives a precise definition of the problem and of the criteria of quality, and it provides instances on which a new engine can be tested and compared.

The formulation of that third track distinguishes four hard constraints, which impose that every lecture is placed, that a room receives one lecture at a time, that courses sharing students or a teacher do not overlap, and that the unavailability of a course is respected; and four soft constraints, which penalise a room too small for its audience, a course concentrated on too few days, isolated lectures inside the day of a group of students, and a course scattered across several rooms. The model of the present project follows the same division and adds what that formulation does not cover : the hierarchy between a promotion and its subgroups, the typing of the rooms, and the calendar of the institution.

Software solutions also exist, such as UniTime or Scientia. They solve the general problem correctly but do not represent the CM, TD and TP hierarchy of the LMD system nor a lunar calendar. Adapting them would require more work than developing an application designed for these rules.

2.5 The choice which follows the generation

The study of the manual practice showed a second difficulty, which the description above does not cover. When the person in charge finally obtains a timetable without conflict, the work is presented as finished, but nothing establishes that this arrangement is a good one. Another arrangement, equally valid, could leave the students two free hours less each week, or gather the sessions of a part-time teacher on fewer days. The person in charge has neither the time nor the means to produce a second arrangement in order to compare.

This is a second problem, of a nature different from the first. Placing the sessions is a question of satisfaction : a rule is respected or it is not. Choosing among several valid timetables is a question of compromise between criteria which cannot all be improved at the same time, since reducing the idle hours of the students often increases the days of presence of the teachers. A method which guarantees the first says nothing about the second.

The consequence for the design of the application is direct. The system must contain two parts : one which decides and must never be wrong, and one which compares, ranks and explains, whose errors cost a worse ranking and not an unusable timetable. This separation determines the whole of the technical study which follows, and in particular the place given to the methods which learn.

3. Study of the datasets

3.1 Approach followed

An engine of this kind cannot be developed without data. Before writing any code, the public datasets available for timetabling were searched, downloaded, and above all opened and verified. The verification was done by reading the files themselves and not their documentation, because a dataset can be described as complete and contain errors.

Each source was examined against the same grid, so that the conclusions can be compared and so that the reason for rejecting a source is stated and not implied.

- Which entities the source contains : teachers, students, courses, rooms, slots, enrolments.
- Whether the content is valid, that is whether the file respects the rules which it claims to describe.
- Whether its identifiers can be matched with those of another source.
- Whether it distinguishes the sessions of type CM, TD and TP.
- Whether it describes a hierarchy between a promotion, its groups and its subgroups.
- Whether it contains a calendar, holidays or a shortened-day period.
- Whether it contains availability declared by the teachers.
- Under which conditions it may be used in a project of this kind.

3.2 Datasets identified

Table 1. Datasets identified and their content

Dataset	Origin	Content
ITC-2007, Track 3	International timetabling competition of 2007	Curriculum-based course timetabling. Courses, curricula, rooms with their capacity, unavailability of the courses. Held as a repository which bundles the instances with a solver.
XHSTT-2014	Benchmarking project for high school timetabling	Instances described in XML with an explicit catalogue of constraint types and an evaluator.
Kaggle, University Exam Scheduling	Public dataset published in 2024	Examination scheduling data : students, teachers, courses, rooms, slots and enrolments in CSV format.
ITC-2007, Track 1	Same competition, examination track	Examination timetabling instances. Identified for the second increment, not yet opened.

3.3 Problems found during the verification

The sources do not have the same level of reliability, and the verification revealed problems which change the way they can be used.

- **ITC-2007, Track 3** : the files are clean and correspond to their description. The dataset contains no individual student, only the number of students following each course, and it does not distinguish the types CM, TD and TP. What is held is a source repository which bundles the instances together with a solver written by a third party, and the documentation of the project states this provenance rather than presenting the folder as the competition archive itself.
- **XHSTT-2014** : the files are complete and describe many types of constraints, but they concern secondary schools and are written in a form which is heavy to read. Their interest lies in the list of the types of constraints rather than in their content. The number of instances and of countries announced varies between the versions of the archive, so the documentation of this project cites the catalogue of constraint types, which was read directly, and not a count taken from a summary.
- **Kaggle** : two files are not usable. The file of timetables contains rooms occupied twice at the same moment and teachers placed in two rooms at the same hour; the file of slots contains lines whose hour of end precedes the hour of beginning. Only the list of rooms and the enrolments of the students can be kept. The dataset concerns examinations, which places it on the examination side of the project.
- **ITC-2007, Track 1** : identified but not yet opened. Its verification is placed at the beginning of the second increment, and no property is assumed of it before that verification.

3.4 Why the datasets cannot be merged

The first idea was to bring the sources together in a single database in order to obtain a complete set of data. The verification showed that this is not possible, for three reasons.

1. The identifiers do not correspond. A teacher is designated by a code in one dataset, by a long identifier in another, and by a name in the third. Nothing allows one to say that two records designate the same person.
2. The level of detail is different. The competition dataset reasons on courses and gives only the number of students, while the Kaggle dataset gives each student individually. Merging the two would require inventing the missing information.
3. Two files of the Kaggle dataset are invalid. Introducing them into a common database would mean placing conflicts inside the data which is supposed to serve as reference.

Merging would therefore have produced a set which is neither a valid benchmark nor realistic data. Each source is instead given one role, and the application runs on data produced for it.

3.5 Why a generated dataset is necessary

Beyond the question of merging, none of the sources contains what the project needs most.

- None distinguishes the sessions of type CM, TD and TP.
- None describes the hierarchy between a promotion, its groups and its subgroups.
- None contains the Tunisian calendar nor a shortened-day period.
- None contains availability declared by the teachers, since this is precisely the information which the application will collect.

A dataset representing a Tunisian faculty must therefore be produced for the project. This is a normal situation in this field, on condition that the generated data is not invented at random and that it is verified before use.

3.6 Method of generation and content obtained

The generation follows the rules of organisation of a faculty rather than a random draw. The programmes are declined by level according to the LMD structure. Each promotion is divided into groups whose size corresponds to the usual size of a tutorial group, and each group is divided into subgroups for the laboratory sessions. The sessions are then created from the courses : one lecture for the promotion, one tutorial for each group, one laboratory session for each subgroup. The rooms are typed as lecture theatre, classroom, computer laboratory or science laboratory, with capacities consistent with the size of the groups which use them. The calendar reproduces the periods of a working day, the holidays of the year and the shortened-day window.

The instance obtained is held in thirteen files. Its full content — programmes, promotions, groups, students, courses, sessions, teachers, rooms, the slot grid and the calendar — is specified in the Cahier des Charges, section 8.2, which is the reference for every count cited elsewhere in the documentation.

The availability file is populated with generated declarations and each line carries the value SYNTHETIC in its source column. This point matters for the honesty of the demonstration : the application collects availability from a web form, and the generated declarations only make the instance solvable before any teacher has connected. The column allows a generated declaration and a real one to be told apart at any moment.

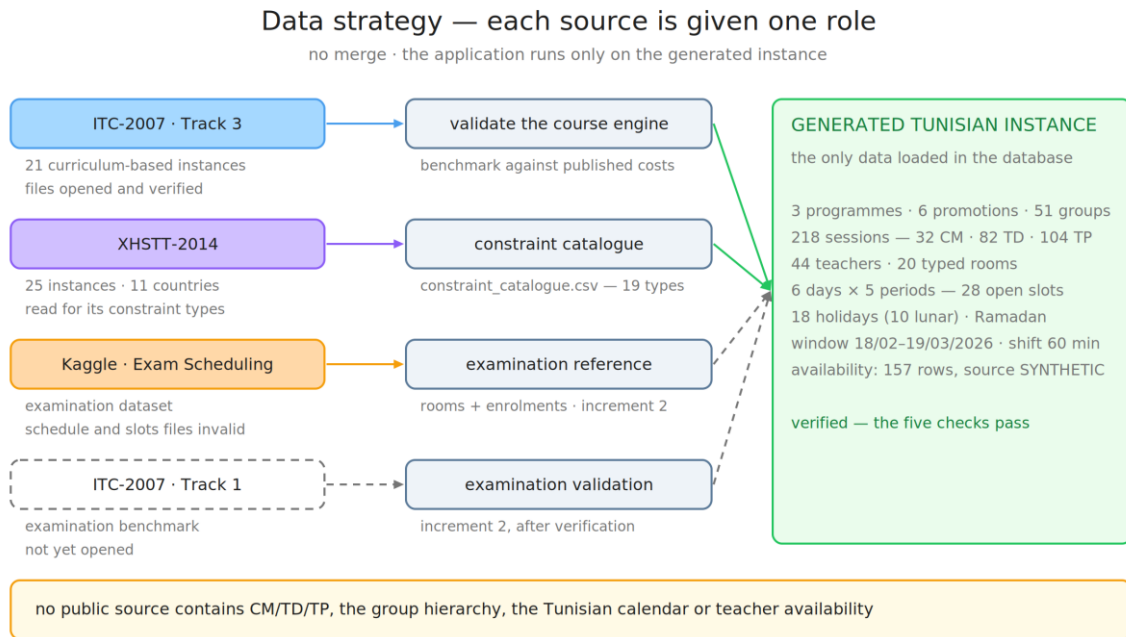


Figure 1. Role given to each source and origin of the data used by the application.

3.7 Role finally given to each source

Table 2. Role finally given to each source

Source	Role	Loaded into the database
ITC-2007, Track 3	Validation of the course engine against published results	No
XHSTT-2014	Catalogue of the types of constraints to be supported	No, transcribed into constraint_catalogue.csv
Kaggle, exam scheduling	Reference for the examination model, rooms and enrolments only	No, increment 2
ITC-2007, Track 1	Validation of the examination engine	No, increment 2
Generated instance	Data on which the application runs	Yes

3.8 Data required by the adjustment of the weights

The application must not only produce timetables, it must rank them, and the order depends on weights given to the criteria of quality. The question therefore arises of where those weights come from, and it is the same question as the previous one : which data is available.

The answer is that none is. Adjusting the weights automatically would require a collection of timetables accepted or refused by a department, with the reason for the decision. No such collection exists for a Tunisian faculty, and it cannot be reconstructed after the fact, since the published timetables of past years carry no trace of the alternatives which were set aside.

Two consequences follow, and both are assumed. The weights start at the values held in the catalogue, which are the only ones available at the beginning and which already give a usable order. And the application becomes, from its first use, the instrument which produces the missing data : each time the person in charge examines two candidates and retains one, that choice is

recorded as a comparison. After a sufficient number of such comparisons, an adjustment of the weights becomes possible, and it is evaluated before being adopted. This is the object of section 4.7.

4. Choice of the resolution method

4.1 Nature of the problem

The problem has two levels which must both be treated. The first is a question of satisfaction : the timetable must respect a set of rules which admit no exception. The second is a question of quality : among the timetables which respect these rules, some leave fewer idle hours to the students or fewer days of presence to the teachers. The method chosen has to answer both, and above all it must never propose a timetable which breaks a rule, since such a timetable cannot be published.

4.2 Criteria of evaluation

Four criteria were used to compare the approaches. They are stated here before the comparison, so that the decision does not rest on an impression.

- **Guarantee of validity** : the method must not be able to return a timetable containing a conflict.
- **Detection of impossibility** : when no timetable exists, the method must be able to establish it instead of searching indefinitely.
- **Data required** : the method must work with the data of the department, without a base of examples to learn from.
- **Effort of implementation** : the model must be able to be written and adjusted within four weeks.

4.3 Approaches studied

Metaheuristics. Genetic algorithms, simulated annealing and tabu search build a timetable then modify it progressively in order to reduce the number of conflicts and improve the quality. They are frequently used for this problem and are relatively simple to program, which is their main advantage. Their weakness concerns the first criterion : nothing guarantees that the timetable obtained at the end contains no conflict, and when the algorithm finds nothing it is impossible to distinguish a problem without solution from a search which was too short. For an application intended to be used by a department, this uncertainty is disqualifying.

Integer linear programming. This approach expresses the problem in the form of a system of equations and inequalities. It does guarantee validity and can establish that no solution exists, so it satisfies the two first criteria, and the published work on curriculum-based timetabling shows that it produces optimal solutions and useful lower bounds. Its disadvantage is the way it expresses the rule of non-overlap, which requires auxiliary variables and large constants. The model becomes heavier as the number of sessions grows, and the writing of the constraints of hierarchy between promotions and subgroups is less direct. The approach was therefore not retained, without being rejected as incorrect.

Approaches based on learning. Using a neural network to produce timetables would require a large set of validated timetables to learn from. This set does not exist for Tunisian faculties, which

alone eliminates the approach in the framework of this project. A trained model also gives no guarantee that its result respects the rules, which brings back the difficulty of the metaheuristics.

Constraint programming. This approach consists in declaring the variables of the problem, in declaring the rules which link them, and in leaving to a solver the task of finding an assignment which satisfies all of them. The rules are written directly, in a form close to the way the department states them, which makes the model readable and easier to correct. Validity is guaranteed by construction, since the solver never produces an assignment violating a rule which was given to it. The solver is also able to establish that no solution exists, and it accepts a limit of time, returning the best result obtained when the limit is reached.

4.4 Comparison and decision

Table 3. Comparison of the resolution approaches

Approach	Validity guaranteed	Detects impossibility	Data required	Decision
Metaheuristics	No	No	None	Rejected
Integer linear programming	Yes	Yes	None	Not retained
Learning	No	No	Set of examples	Rejected
Constraint programming	Yes	Yes	None	Retained

Constraint programming is the only approach which satisfies the four criteria. It was therefore retained for the assignment of the sessions. This decision concerns the first of the two problems identified in section 2.5, and it does not settle the second, which is treated in sections 4.6 and 4.7.

4.5 Choice of the library and consequences on the solving

The CP-SAT solver of the OR-Tools library was chosen to implement this approach. It is free and maintained, it offers a Python interface which allows it to be called directly by the server, and it accepts a limit of time. Its documentation provides constructions adapted to problems of scheduling, in particular the constraint of non-overlap which applies to intervals of time.

This choice has a counterpart which must be stated. The quality of the result depends on the way the model is written : a badly formulated constraint does not give a false result but can make the search considerably slower. Part of the work of the project consists precisely in writing this model correctly, which is why five days of the planning are devoted to it.

The library also offers a mechanism which the project uses to report the impossible situations. When the constraints are declared under enforcement literals and those literals are given to the solver as assumptions, the solver returns, on an infeasible problem, a subset of those assumptions sufficient to explain the infeasibility. The application therefore does not merely announce that no timetable exists, it names the rules which are in conflict.

Two properties of that mechanism have a direct effect on the architecture, and ignoring them would have produced a defect discovered late.

- **Solving under assumptions does not admit parallelism** : the number of workers must be set to one. The diagnosis run is therefore markedly slower than a normal solving and cannot be used as the ordinary path.
- **The presence of an objective empties the mechanism of its usefulness** : with a function to minimise, the whole set of assumptions is returned rather than a reduced subset. The diagnosis run therefore removes the objective.
- **The subset returned is reduced but not guaranteed to be minimal** : the application presents it as a set of rules sufficient to explain the conflict, and not as the smallest such set. Obtaining a minimal set would require minimising the sum of the literals instead, which is recorded as a possible improvement.

The consequence is that the generation is organised in three stages : an arithmetic pre-analysis which needs no solver, an optimisation run which uses all the workers and carries the objective, and a diagnosis run entered only when the optimisation run concludes that no timetable exists. These three stages are described in the Software Requirements Specification and appear in *Figure 3*.

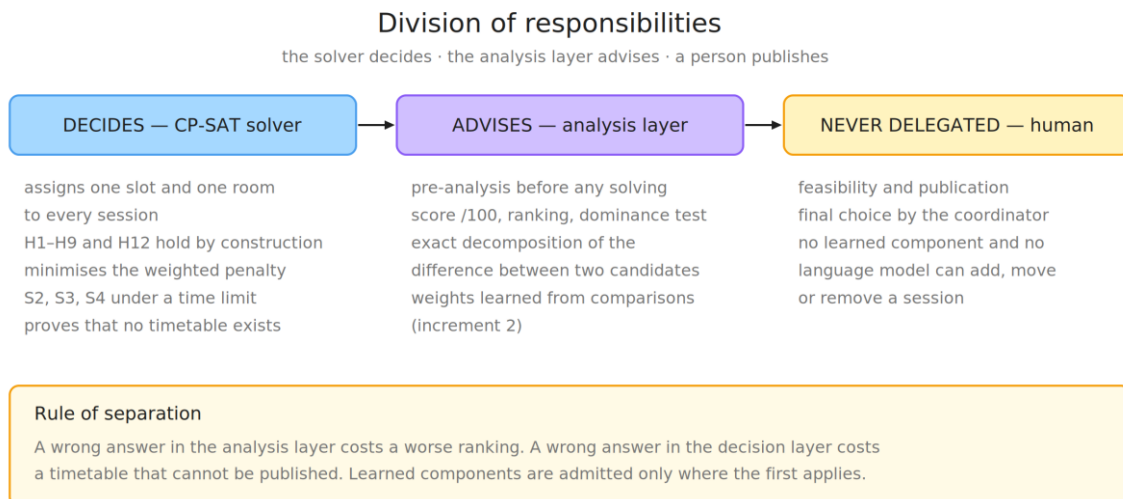


Figure 2. Division of the responsibilities between the decision layer and the analysis layer.

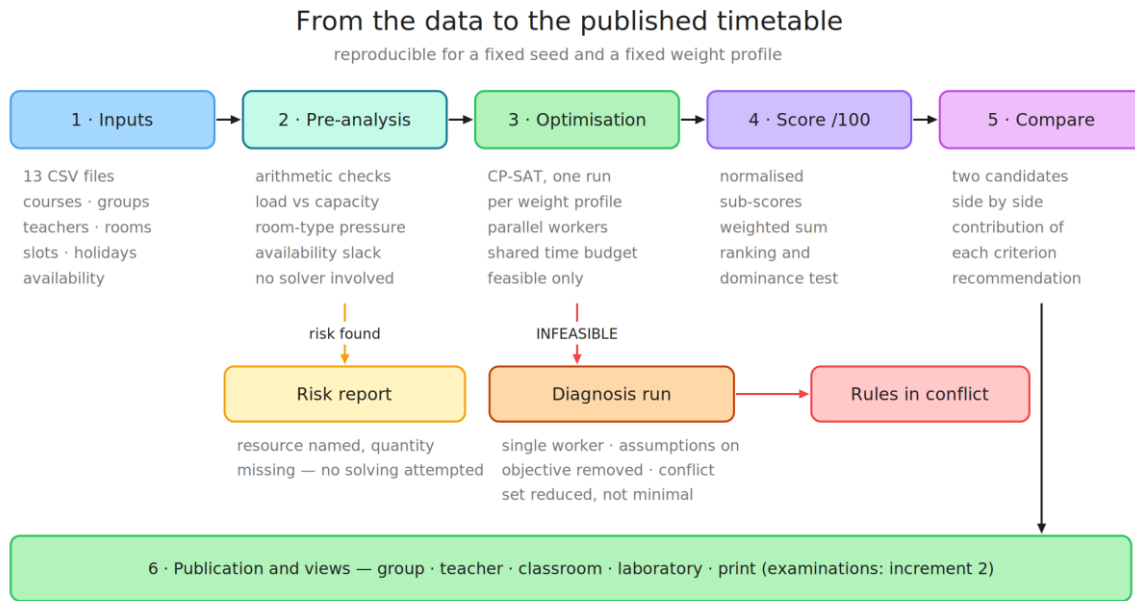


Figure 3. Sequence from the data to the published timetable, with the three stages of the generation.

4.6 Method for the ranking of the valid timetables

The second problem stated in section 2.5 is a choice among candidates which all respect the rules, evaluated on several criteria which cannot be improved simultaneously. Four methods were examined.

Weighted sum of the normalised criteria. Each criterion is brought back to a scale between zero and one against bounds computed from the instance, then the criteria are combined with weights whose sum is one. The result is a score which the user can recompute by hand. Its determining property is linearity : the difference between the scores of two candidates is exactly the sum of the contributions of each criterion, so the explanation shown to the user is the calculation itself. The published formulation of the curriculum-based competition evaluates its timetables in the same way, which places this choice in line with the reference practice of the field.

Lexicographic order. The candidates are compared on the first criterion, then on the second only in case of equality. The method is simple but does not describe the way a department reasons, since a small loss on the first criterion is never compensated by a large gain on the second.

Outranking methods. These methods compare the candidates in pairs using thresholds of indifference and of veto. They are richer, but they require parameters which are harder to justify before a department than a weight, and the result is not a number which the user can verify.

Dominance alone. A candidate improved by another on every criterion can be eliminated without any weight. This test is exact and requires no parameter, but it does not produce an order, since most candidates improve on one criterion and lose on another.

Table 4. Comparison of the methods for ranking the candidates

Method	Produces an order	Verifiable by hand	Parameters to justify	Decision
Weighted sum	Yes	Yes	The weights	Retained

Method	Produces an order	Verifiable by hand	Parameters to justify	Decision
Lexicographic order	Yes	Yes	The order of the criteria	Not retained
Outranking	Yes	No	Thresholds and vetoes	Not retained
Dominance alone	No	Yes	None	Retained as a complement

The weighted sum was retained for the order, and the test of dominance was retained as a complement : when the candidate placed first is improved by another on every criterion, the interface signals it, because that situation reveals that the weights are hiding a compromise rather than expressing one.

4.7 Place of the learning

The weights of the criteria express what the department considers a good timetable, and there is no reason for those of one faculty to suit another. They are therefore fixed at the beginning from the catalogue, and then adjusted from the comparisons which the person in charge makes on the comparison screen, in the conditions described here.

Learning the weights of a decision maker from the comparisons that the decision maker makes is an established method of multi-criteria decision analysis, where it is treated as a problem of learning to rank : the pairs retained constitute the training information, and the weights obtained can then predict the choices of the same decision maker on new alternatives. The project applies that method in its simplest form.

Each comparison provides a pair of candidates and the one which was retained. The vector of the differences between their normalised criteria is computed, and a logistic regression without constant term is fitted on these vectors, so that the probability of preferring a candidate increases with the difference of its score. The weights obtained are exactly the coefficients of this regression.

Three restrictions are imposed on the fitting, and each answers a precise objection.

- **Non-negative weights** : reducing the number of violations of a criterion must never lower the score. Without this restriction a coefficient could become negative on a small sample, and the application would then reward a defect.
- **Weights whose sum is one** : the scores of two runs must remain comparable. Multiplying all the weights by the same positive number does not change the order of the candidates, so this normalisation fixes the scale of the score without modifying any ranking.
- **Regularisation towards the weights in force** : with few examples the fitted values must not move far from those of the catalogue, which remain the reference as long as the evidence is thin.

A larger model was considered and set aside. The number of criteria is of the order of seven and the number of comparisons available before the end of the project is of the order of a few tens. A linear model is the largest model which such a sample supports, and it is also the only one which

preserves the exact decomposition of the difference between two scores, which is the property on which the whole explanation rests.

The adjustment is adopted only if it proves better than what it replaces. The comparisons collected are evaluated by leaving one out at a time and predicting it with the weights fitted on the others. If the accuracy so obtained does not exceed that of the weights in force, those are kept and the adjustment is reported as not established. This criterion is stated before the experiment and can be failed, which is the condition for the result to mean anything.

4.8 Place of the language model

The project integrates an external language model through an API. The question is not whether to use one, since the internship notes require it, but which tasks may be given to it. Four uses were examined and separated by a single test : can the output be checked?

Table 5. Uses of a language model examined

Use	Output checkable	Consequence of an error	Decision
Explain a candidate, compare two, answer a question, write a report	Yes, every figure must appear in the context supplied	A sentence to be corrected	Retained
Suggest an improvement	Yes, once mapped onto the recommendation catalogue	A recommendation which is refused	Retained, mapped
Compute the score or decide the ranking	No, the figure would come from the model	A number nobody can recompute	Rejected
Translate a sentence into a constraint	Only with a solver validating the generated model in a loop	A timetable wrong without being detectably wrong	Out of scope

4.8.1 Why scoring and ranking stay outside the model

The notes ask the AI to score and to rank. Both are arithmetic operations : the department must be able to recompute a score by hand and obtain the same order twice from the same data. A language model gives neither guarantee. The platform therefore computes the score and the order by the weighted sum of section 4.6, and the model explains the result, answers questions about it and writes it up. The requirement is satisfied in substance, and the numbers stay verifiable.

4.8.2 Why natural-language constraint entry stays outside

Recent evaluations of language models on constrained problems show that their performance is highly sensitive to the way a problem is presented, and that they remain fragile when constraints interact strongly or when the formulation departs from familiar templates. Work which does succeed in translating a description in ordinary language into a constraint model obtains its results by validating the generated model with a solver in a loop; the reliability comes from that validation, not from the model. Building such a harness does not fit the duration of the project. The reason recorded is the absence of a verification, not a general distrust.

4.8.3 Grounding

The retained uses are all read-only and all grounded. A context builder assembles, from the run, the figures needed to answer the question. The model receives that context and the question, and nothing else. The answer is then checked : every number appearing in the text must appear in the context. If the check fails or the service is unreachable, the computed form is displayed instead.

The check catches invented figures. It does not certify that the sentence is right about everything else, and the documentation says so rather than claiming more. That limit is precisely why no decision the department must defend depends on this service.

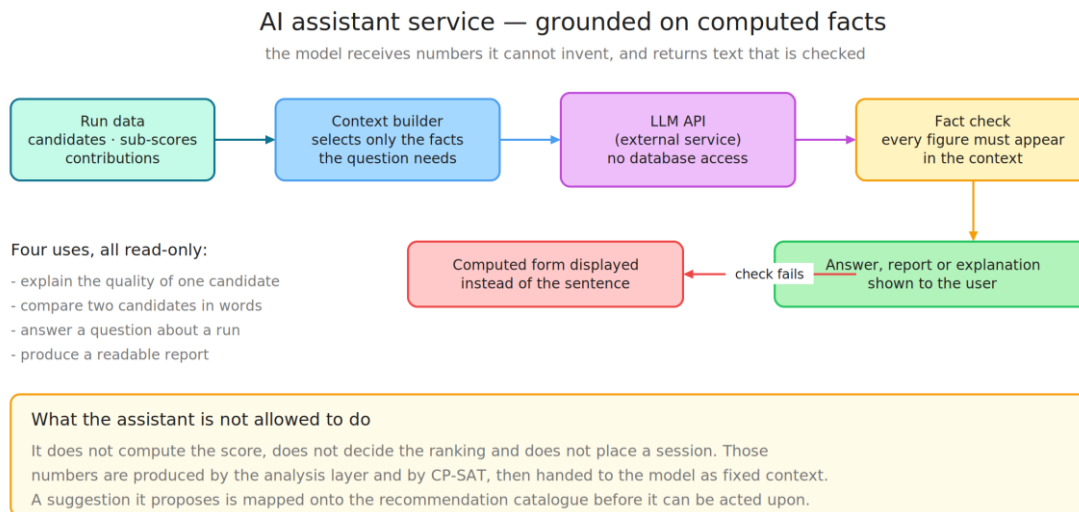


Figure 4. AI assistant service, grounded on computed facts

4.9 Recommendation-driven regeneration

The internship notes ask for a closed loop : the AI analyses a candidate, proposes an improvement, and, if the person in charge accepts it, the application rebuilds an improved timetable automatically. Read literally, this could be understood as the AI editing a timetable directly, which would break the separation stated in section 2.5. The loop is therefore built the other way round.

A recommendation is never a timetable edit. It is a change to one of the three inputs the solver already accepts :

- **a weight (weight_delta)** : raise or lower the weight of one soft criterion for the next run;
- **a lock (lock_session)** : fix one session to its current slot and room, so it is excluded from the next search;
- **an exclusion (exclude_slot)** : remove one slot or one room from the domain of one session before the next run.

Accepting a recommendation therefore does not call a new component. It calls the same solving step described in section 4.5, with one input changed. The marginal engineering cost is small because the portfolio mechanism already loops over several inputs to produce several candidates; a recommendation simply adds one more input to that loop. The guarantee carries over for the same reason it held for the first candidate : the timetable is not touched, only the instructions given to the solver are.

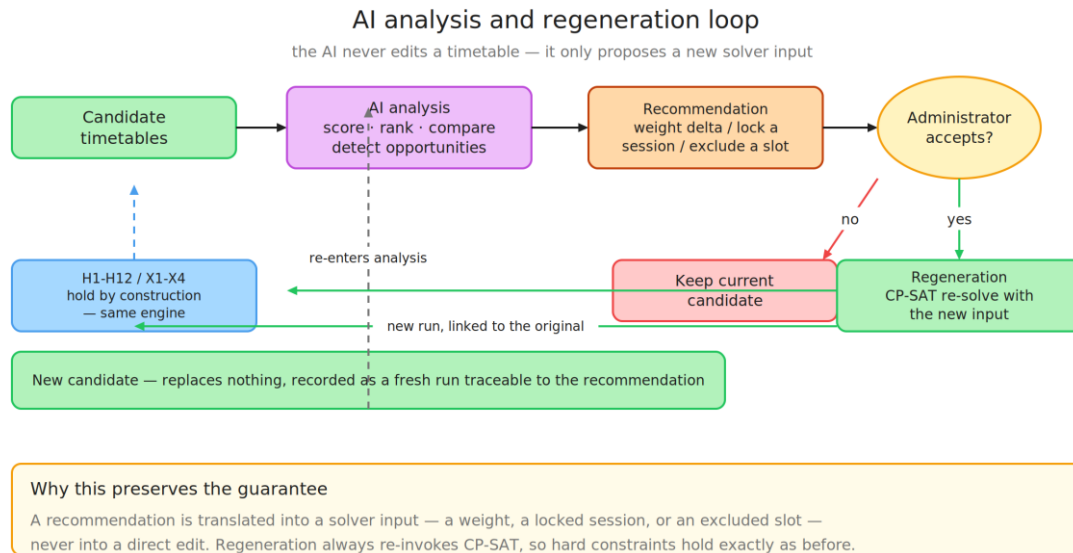


Figure 5. AI recommendation and regeneration loop.

The catalogue of expressible actions is kept deliberately small. A recommendation which cannot be expressed as one of the three actions above is not offered to the person in charge, rather than being approximated. This is the same discipline applied in section 4.8 to the language model : the system proposes only what it can carry out through the solver, and nothing that would require an unverified edit.

5. Choice of the technical architecture

The system is organised in four layers — presentation, application, decision, analysis — specified in the Software Requirements Specification, section 2.1. This section explains the choice of technology behind each one.

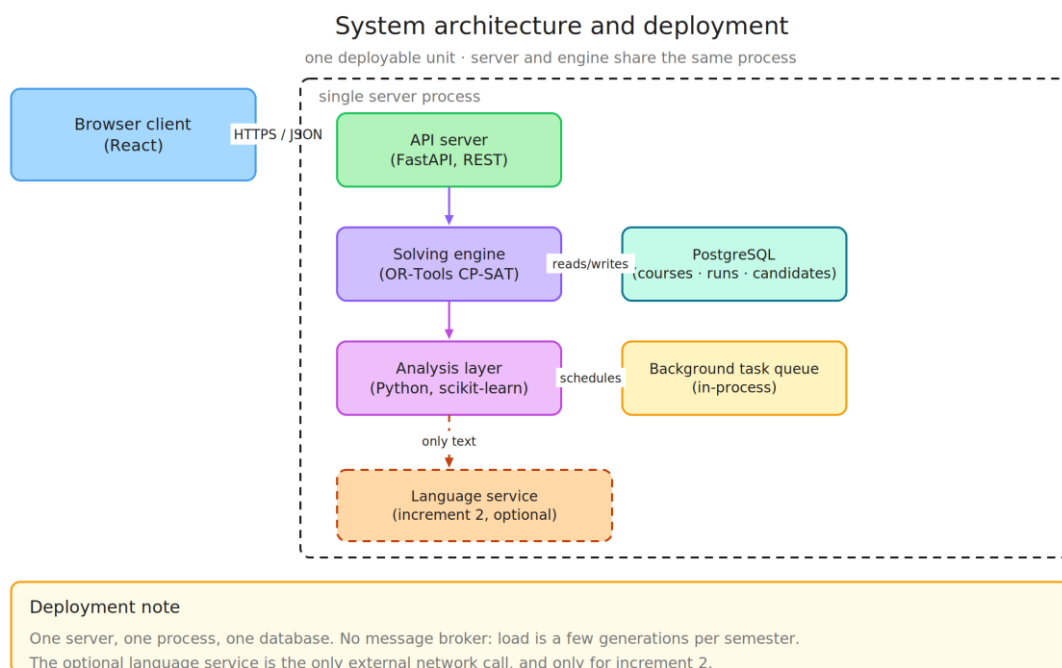


Figure 6. System architecture and deployment.

5.1 Server

The solving engine being written in Python, the server was written in the same language so as to call it directly, without exchange between two separate programs. Among the Python frameworks, Django offers an administration interface and a system of templates which are of no use here, and Flask would have required adding the validation of the data by hand. FastAPI was retained because it provides this validation and generates the documentation of the interface automatically.

5.2 Database

The data of the problem is strongly linked : a session refers to a course, a group, a teacher and a room, and these links must remain consistent when several users modify the data at the same time. PostgreSQL was retained for its guarantees on this point. MySQL would have been acceptable but offers weaker guarantees, and SQLite, which is a single file without real concurrent access, does not suit an application used by several people at the same time.

5.3 Interface

The interface displays weekly grids which have to be redrawn when the user changes a filter, and a comparison screen which places two candidates side by side. React was retained because its organisation in components corresponds to this type of display and because calendar components are easy to find. Vue would have been an equally valid choice, and this decision is not determining for the project.

5.4 Organisation of the application

A division into several independent services was considered and set aside. That organisation is useful when several teams work in parallel or when parts of the system must be enlarged separately, which is not the case here. A single application is simpler to develop, to test and to install, and it allows the server and the engine to share the same execution environment, which avoids transmitting the whole data of the problem between two programs at each generation.

5.5 Institutional rules held as configuration

The shortening of the working day during Ramadan and the closing of certain half-days could have been written as constraints of the model. They are not, and the reason is an engineering one rather than a matter of convenience.

A rule written into the model cannot be changed by the department. A rule held as data can. These rules are decided by the institution, they differ from one establishment to another and they change from one year to the next; the dates of Ramadan move by about eleven days each year, and which half-days are closed is a local decision. Writing them into the model would oblige the department to call a developer in order to apply a decision that it has itself taken.

The mechanism retained uses what the model already contains. A closed half-day sets the field `is_open` of the corresponding slots to zero, and the constraint H9, which forbids the closed slots and the holidays, removes them from the domain of every session. A shortened-day period shifts the start and end hours of the periods by a configured number of minutes; the index of the slot does not change, so the model is untouched and only the hours displayed differ. No new constraint is written, and the behaviour of the application changes with the configuration alone.

The limit of this choice is stated as clearly as its advantage. A rule held as data can be modified by anyone who holds the administration rights, including by mistake. If the institution confirms in writing that a rule of this kind must never be violated under any circumstance, it is promoted to a hard constraint of the model and receives its own code in the catalogue. Until that confirmation, configuration is the safer of the two.

5.6 Execution of the generation

The application produces several candidates under distinct weight profiles, and the total duration of a generation is counted in minutes. A request from the browser cannot be held open for that time, and the diagnosis run described in section 4.5 is slower still, since it uses a single worker.

The organisation retained is the simplest which answers this. A run is recorded in the database with its state, its parameters and its seed. A task executed in the background takes it, solves the profiles one after the other, and records each candidate as soon as it is obtained. The interface asks for the state of the run at regular intervals and displays the candidates as they appear.

A queue served by a separate broker was considered and set aside for this project. It would bring a component to install, to secure and to monitor, for a load of a few generations per semester on a single server. The decision would have to be reconsidered if several departments generated at the same time, and it is recorded here so that it can be.

5.7 Isolation of the analysis components

The components which score, order and explain receive the candidates as data and return numbers. They hold no reference to the solver and have no right to write in the timetables. This separation is not a matter of presentation, it has two consequences which the project uses.

- They can be tested alone, on candidates written by hand, without executing any solving.
- A defect in them can produce a wrong ranking but can never produce an invalid timetable, which places the whole risk of these components on a part of the system where an error is visible and correctable.

6. Development methodology

6.1 General approach

The work is organised in successive phases, each one producing a result which the following one uses. No phase begins on suppositions : the study of the data precedes the choice of the method, and the specification precedes the implementation. This order was chosen because a correction made late in this type of project is expensive, whereas a verification made early costs little.

6.2 Phases of the work

The reason for each phase is given, since a phase which cannot be justified does not deserve to appear in the planning.

- **Understanding of the problem** : identify what makes the manual construction long and what the application must replace.

- **Study of the existing work** : examine the competitions, the published instances and the existing software, in order not to redo what already exists and to reuse the criteria of quality already defined.
- **Analysis of the data** : open and verify the available datasets, determine what they contain and what is missing. This step conditions everything that follows, since an engine cannot be tested without data.
- **Engineering of the requirements** : transform the observations into requirements which can be verified, each one associated with a criterion of acceptance.
- **Cahier des Charges** : fix what the application must do and what it will not do, so that the scope stops moving during the development.
- **Specification of the requirements** : express each requirement in terms of data, interfaces and constraints, in a form directly usable for the implementation.
- **Design of the database** : define the entities and their links, in particular the hierarchy of the groups without which the rule H12 cannot be written, and the record of the runs without which no result can be traced.
- **Modelling of the constraints** : declare the variables and the rules in the language of the solver. This is the central phase of the project.
- **Scoring, ranking and explanation** : compute the sub-scores, order the candidates and decompose the difference between two of them.
- **Development of the server** : expose the operations of loading, of generation, of comparison and of consultation, and connect them to the engine.
- **Development of the interface** : build the grid of availability, the comparison screen and the views of the timetable, which are the parts seen by the users.
- **Tests and validation** : verify the engine on the published instances, verify the properties of the analysis layer, and verify the requirements one by one against their criteria of acceptance.
- **Documentation and preparation of the presentation** : assemble the documents produced and prepare the presentation of the work.

6.3 Delivery in two increments

The scope requested exceeds what four weeks allow, and the work is therefore divided into two increments whose contents are fixed in advance. The first is complete in itself : it produces a usable application, and it is the object of the commitment taken in the Cahier des Charges. The second is specified and designed with the first, so that it can be undertaken without a new study.

Table 6. Delivery in two increments

Increment	Condition of completion	Estimated duration
Increment 1	A department can load its data, collect availability, obtain several candidates, compare them, read the reason for their order, ask the assistant about them, accept a recommendation to regenerate an improved one, and publish	20 days
Increment 2, examinations	The timetable of an examination session is produced under the constraints X1 to X4	4 days

Increment	Condition of completion	Estimated duration
Increment 2, weights	The adjustment of the weights is implemented and evaluated against the weights in force	3 days
Increment 2, natural-language constraints	Not undertaken. Recorded here as the use which would require a solver-validation harness	not estimated

Announcing the second increment as committed would place the project in the situation which section 8.3 is meant to avoid : a scope which cannot be reduced without breaking a promise. It is therefore stated as conditional from the beginning.

7. Validation strategy

The validation rests on four verifications of a different nature, each answering a distinct question.

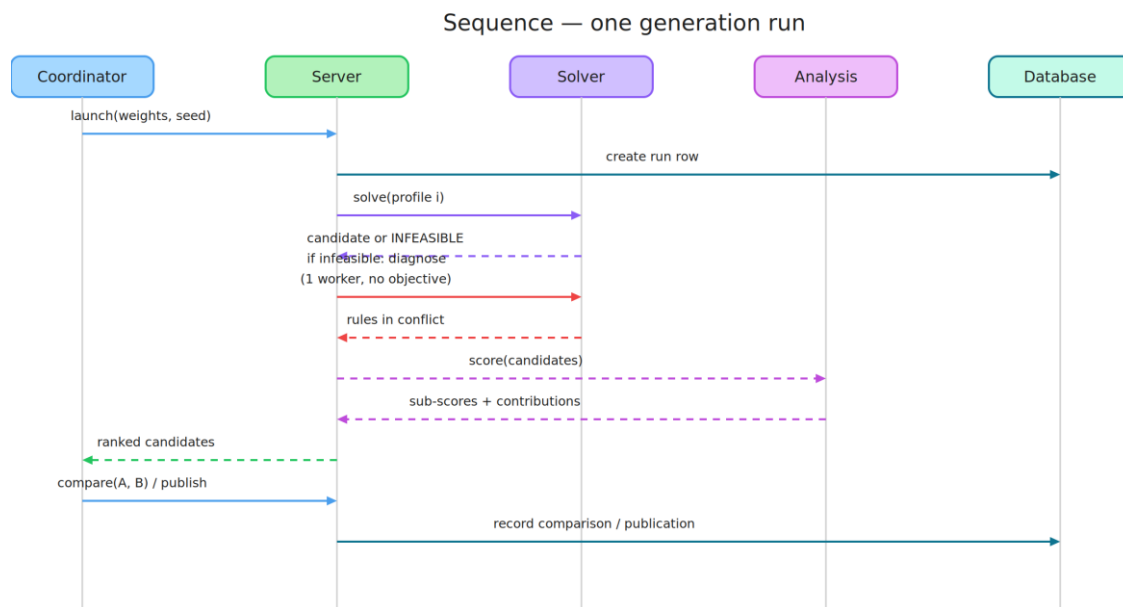


Figure 7. Sequence of one generation run, from launch to publication.

7.1 Validation on the published instances

The engine is tested on the instances of the curriculum-based track of the competition of 2007, for which results are published. This verification answers the question : is the model correct? Two points are examined, the absence of any violation of a hard constraint in the timetables produced, and the distance between the cost obtained and the best known results.

The objective is not to obtain better results than those published, which would not be realistic in four weeks, but to show that the engine produces valid timetables of a reasonable quality on instances which it did not serve to build. For the examination module, the same procedure is applied to the instances of the first track, after their verification.

7.2 Verification of the generated instance

Five verifications are applied to the data before any solving. They answer the question : is the instance solvable? Their purpose is to distinguish a problem which genuinely has no solution from an error in the model, which are two situations that a solver reports in the same way. The results obtained on the instance supplied are given in the Cahier des Charges.

- Each session can find a room of the required type and of sufficient capacity.
- The number of open slots covers the demand for each type of room.
- No teacher is assigned more hours than the maximum load of the rank.
- Each teacher keeps enough free slots for the sessions assigned.
- The hierarchy of the groups is consistent and the students match the declared group sizes.

The second verification is the one to watch : on the instance supplied, the computer laboratories are occupied at 95 per cent of the available capacity, so the withdrawal of a single laboratory would make the instance very probably infeasible. The pre-analysis reports this figure before any solving, which is precisely its purpose.

7.3 Functional tests

The functions of the application are tested one by one on the generated instance. This verification answers the question : does the application do what the Cahier des Charges announces? Each requirement is verified against its criterion of acceptance, which was written at the same time as the requirement itself.

7.4 Validation of the analysis layer

The components which score, order and explain are verified by properties rather than by examples, because a property holds for every input while an example holds for one. Four properties are tested on candidates generated at random.

- **Exactness of the decomposition** : the sum of the contributions displayed for a comparison equals the difference between the two scores. This identity is a consequence of the linearity of the score and its failure would reveal an error in the calculation.
- **Invariance of the order** : the same candidates, the same weights and the same bounds give the same order, whatever the order in which the candidates arrive.
- **Monotonicity** : reducing the violations of one criterion, all else being equal, never lowers the score. This property follows from the non-negativity of the weights and is tested because the fitting could break it.
- **Fidelity of the formulation** : every figure appearing in the sentence produced by the language model appears in the structured explanation which was given to it.

The adjustment of the weights is evaluated separately, by leaving out one comparison at a time and predicting it with the weights fitted on the others. The result is compared with the accuracy of the weights in force, and the adjustment is adopted only if it exceeds it.

7.5 Criteria of acceptance

Table 7. Criteria of acceptance of the validation

Verification	Question answered	Expected result
Published instances	Is the model correct?	No violation, cost of a reasonable order
Verification of the data	Is the instance solvable?	The five verifications pass
Properties of the analysis layer	Is the ranking exact and stable?	The four properties hold on the random candidates
Evaluation of the weights	Does the adjustment improve on the weights in force?	Accuracy higher, otherwise the weights in force are kept
Functional tests	Does the application respond to the requirements?	Each criterion of acceptance verified

8. Planning

8.1 Phases and duration

The project has twenty working days. The distribution below gives the largest part to the modelling of the constraints, which is the phase carrying the most uncertainty, and covers the first increment only.

Table 8. Phases and duration

Phase	Work	Duration
Phase 1	Study of the needs, specification, evaluation of the sources, verification of the instance	3 days
Phase 2	Modelling of the constraints H1 to H12 and first valid timetable	5 days
Phase 3	Weight profiles, portfolio, score, ranking, decomposition, recommendation catalogue and regeneration, validation on the published instances	3 days
Phase 4	Grid of availability, screen of generation, screen of comparison and recommendation, views	4 days
Phase 5	Verification before solving, diagnosis run, authentication, rights and record of the runs	2 days
Phase 6	Tests, documentation and preparation of the presentation	3 days

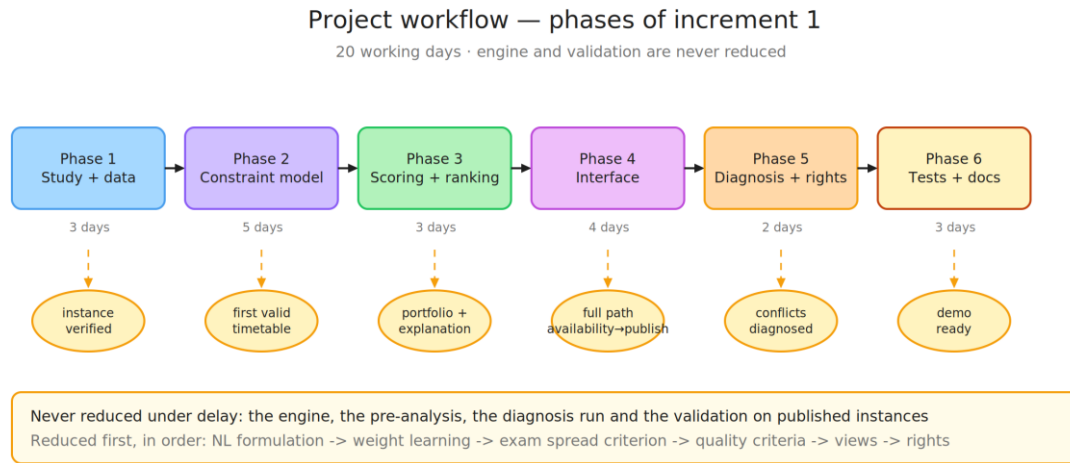


Figure 8. Project workflow, phases of the first increment.

8.2 Milestones

Table 9. Milestones of the project

Milestone	Condition to be reached
End of phase 1	Instance verified, evaluation of the sources written, scope fixed
End of phase 2	A timetable without conflict obtained on the instance
End of phase 3	Several candidates produced, ordered, and the difference between two of them decomposed
End of phase 4	Complete path from the declaration of availability to the publication
End of phase 5	An infeasible instance produces a report naming the rules in conflict
End of phase 6	Application demonstrable and documents completed

8.3 Order of reduction in case of delay

If the work takes longer than expected, the scope is reduced in a determined order, decided in advance so that the decision is not taken under pressure.

4. The report generation of the assistant is abandoned, explanations and answers being kept.
5. The adjustment of the weights is abandoned, the weights of the catalogue being kept.
6. The examination session is reduced to the placement under the constraints X1 to X4, without the criterion of spreading.
7. The number of criteria of quality taken into account in the score is reduced.
8. The number of views displayed is reduced.
9. The management of the rights is reduced to the separation between the person in charge and the other users.

The engine, the verification of the data before solving, the diagnosis of the infeasible instances, the score with its decomposition and the validation on the published instances are never reduced,

because they carry the demonstration that the application works and that its results can be justified.

9. Risks and mitigation

Table 10. Risks and mitigation

Risk	Effect	Mitigation
Modelling of the constraints longer than expected	Delay on the central phase	Begin from the first day and rely on the published instances as examples
Extension of the scope during the work	Nothing is finished	Scope fixed in the Cahier des Charges, second increment conditional, order of reduction decided in advance
Solving time higher than the target	Result too slow to be used	Limit of time imposed, the profiles solved one after the other, the first candidate returned as soon as it is available
Diagnosis run too slow because it uses a single worker	The report of the conflicts makes the user wait	The diagnosis is entered only after an infeasible result, it carries no objective, and its own time limit is separate
Saturation of the computer laboratories, occupied at 95%	Instance without solution after a small change	Pre-analysis reporting the pressure per room type, number of laboratories configurable in the generator
Candidates too similar to be worth comparing	The comparison screen has no content	Profiles chosen far apart and duplicate candidates not presented twice
Too few comparisons collected	The adjustment of the weights cannot be evaluated	Criterion of adoption stated in advance, weights in force kept and the result reported as not established
Confusion between what is guaranteed and what is estimated	A reader trusts a figure which is only an estimate	Each result is presented with the method which produced it, and the separation of the layers is stated in the documentation and in the interface
Absence of real data	Doubt on the realism of the results	Instance generated from the rules of organisation of a faculty, verified by five checks, generated availability marked as such

10. Deliverables

- Solving engine and its validation on the published instances.
- Evaluation of the public sources, with the grid applied and the conclusion retained for each source.
- Generator of the instance and the thirteen files produced, with the report of the five verifications.
- Catalogue of the constraints referenced to the XHSTT types, file constraint_catalogue.csv.
- Components of scoring, ranking and explanation, with the tests of their properties.
- Recommendation catalogue and regeneration module, reusing the solving engine.

-
- AI assistant service : context builder, call to the language model API, and verification of the figures returned.
 - Server and web interface covering the requirements FR-1 to FR-19 and FR-23.
 - Documentation : this document, the Cahier des Charges and the Software Requirements Specification.
 - Figures in editable Excalidraw format.
 - Record of the generation runs carried out during the validation.
 - Presentation of the work.